

Week 2

Full-Stack Business Application Development

Frontend, backend, database, REST APIs and authentication

For M.Sc. Computer Science Students | 30 Hours

Agenda

This session sequence is designed for teaching and live coding.

1

Understand full-stack architecture

2

Finalize project requirements and roles

3

Design database schema

4

Create backend REST APIs

5

Test APIs with Postman

6

Build frontend screens

7

Integrate frontend with backend

8

Add validation, authentication and demo

What you will learn after completing this week

By the end of this week, students should be able to:



Explain client-server architecture



Store and retrieve data from database



Design tables or collections for a business app



Add validation and error handling



Build CRUD REST APIs



Implement basic login and roles



Test endpoints using Postman



Document API endpoints



Connect frontend forms to backend APIs



Demo a full-stack workflow

Project of the Week: Placement Cell Portal

A real business system that directly supports employability.

- Student module: profile, skills, resume link and eligibility details.
- Company module: company profile and job posting.
- Application module: students apply for jobs and track status.
- Admin module: verify data, view dashboard and manage records.
- Authentication module: login for admin, students and companies.
- Analytics extension: count applications, shortlisted candidates and placed students.

Instructor cue: The same architecture can be adapted for library, inventory or attendance systems.

Full-Stack Architecture

Frontend

React or HTML/CSS/JS screens. Sends requests and displays responses.

Backend API

Express or Flask server that contains business logic.

Database

MongoDB, MySQL or PostgreSQL stores persistent records.

Postman

Tests API endpoints before integrating UI.

Auth

Login, role checks and protected routes.

GitHub

Version control, issues, branches and documentation.

Requirement Breakdown and User Roles

Every business app begins with users, permissions and data.

- Admin: manages students, companies, job posts and application status.
- Student: creates profile, views jobs and applies.
- Company: creates profile and posts job openings.
- Mentor/Trainer: reviews progress and placement activity reports.
- Define minimum viable product first; extra features come later.
- Convert every requirement into a route, screen and database entity.

Instructor cue: Ask each team to explain one role and the data that role owns.

Database Design Starter

Entity	Fields	Relationships	Notes
Student	name, email, skills, cgpa, resumeUrl	has many applications	Email should be unique
Company	name, website, sector, contactEmail	posts many jobs	Used for job source
Job	title, skills, package, deadline, companyId	belongs to company	Filter by skill and eligibility
Application	studentId, jobId, status, date	joins student and job	Status: applied, shortlisted, selected
User	email, passwordHash, role	maps to student/company/admin	Used for login

REST API Design

An API is a contract between frontend and backend.

- GET /students - list student records.
- POST /students - create a new student profile.
- GET /students/:id - retrieve one profile.
- PUT /students/:id - update details.
- DELETE /students/:id - remove record if permitted.
- POST /auth/login - authenticate user and return session or token.
- Use consistent response format: success, data, message, error.

Instructor cue: Teach route naming clearly; many student bugs come from inconsistent endpoint names.

Backend Route Example with Express

- Define route with method and path.
- Validate input before saving.
- Use try/catch for database operations.
- Return useful HTTP status codes.

```
app.post("/api/students", async (req, res) => {
  try {
    const { name, email, skills } = req.body;
    if (!name || !email) {
      return res.status(400).json({ message: "Missing fields" });
    }
    const student = await Student.create({ name, email, skills });
    res.status(201).json({ data: student });
  } catch (error) {
    res.status(500).json({ message: "Server error" });
  }
});
```

Postman Testing Workflow

Collection

Create one collection for the project and folders by module.

Environment

Store baseUrl and authToken as variables.

Request Body

Use JSON payloads for POST and PUT requests.

Status Codes

Check 200, 201, 400, 401, 404 and 500 cases.

Tests

Add simple assertions for status and response fields.

Documentation

Export collection or add screenshots to report.

Authentication and Role-Based Access

Teach simple first; strengthen security in Week 3.

- Registration creates a user with role: student, company or admin.
- Login verifies email and password.
- Passwords must not be stored as plain text; hash them.
- Protected routes require a valid session/token.
- Role checks decide which user can access which endpoint.
- Example: only company/admin can create job posts.

Instructor cue: Use clear examples of what each role can and cannot do.

Frontend Screens to Build

Each screen should connect to at least one API endpoint.

- Login/Register page.
- Student profile form and profile list.
- Company registration/profile page.
- Job posting form and job list.
- Application submission and status page.
- Admin dashboard showing counts and recent records.
- Navigation bar with role-based menu items.

Instructor cue: Encourage functional UI before advanced styling.

Frontend API Call Example

- Use fetch or Axios to call the backend.
- Keep API URL in one config file.
- Show loading, success and error messages.
- Refresh list after create/update/delete.

```
async function loadJobs() {
  const response = await fetch(`${API_URL}/api/jobs`);
  const result = await response.json();
  setJobs(result.data);
}

async function createJob(formData) {
  await fetch(`${API_URL}/api/jobs`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(formData)
  });
  loadJobs();
}
```

Validation and Error Handling

Professional applications guide users instead of failing silently.

- Frontend validation: required fields, email format and numeric fields.
- Backend validation: never trust frontend inputs.
- Database validation: unique email, required references and valid status values.
- Consistent error messages help users and testers.
- Use clear HTTP status codes.
- Log server errors without exposing sensitive details to users.

Instructor cue: Ask students to test invalid inputs, not only happy path.

Five-Day Implementation Plan

Day	Focus	Student Task	Evidence
1	Design	Requirements, roles, database schema and API list.	Requirement sheet and schema
2	Backend	Create server, models and CRUD endpoints.	Postman-tested APIs
3	Frontend	Create pages, forms, tables and navigation.	UI screenshots
4	Integration	Connect frontend to backend and database.	Working CRUD demo
5	Auth	Add login, role logic, README and demo.	Full-stack demo package

Lab Tasks for Students

Basic

CRUD for students, companies and jobs.

Intermediate

Application workflow with status update.

Auth

Login and role-based access for screens.

Testing

Postman collection with success and failure cases.

Documentation

API table and database schema in README.

Challenge

Add eligibility filter by skills or CGPA.

Assessment and Weekly Deliverables

Evaluate whether students can build connected software.

- Working full-stack application with database.
- At least 8 documented API endpoints.
- Database schema/table design.
- Postman collection or testing screenshots.
- README with architecture diagram and setup steps.
- Demo showing create, read, update and delete flow.
- Weekly progress report with problems faced and solutions.

Instructor cue: During viva, ask students to trace one action from UI to database.

Study Materials and Reference Links

Use official documentation first. These links are suitable for student practice and instructor preparation.

[React Learn](#)

Components, props, state, effects and UI thinking.

[Mongoose Docs](#)

MongoDB object modeling for Node.js.

[Express Routing Guide](#)

Routes, methods, middleware and request handling.

[PostgreSQL Docs](#)

Relational database documentation.

[Postman Learning Center](#)

API requests, collections, documentation and testing.

[JWT Introduction](#)

Basic token structure for authentication concepts.

[MongoDB Atlas: Connect to Cluster](#)

Cloud database connection and access setup.

[bcrypt npm](#)

Password hashing library used in Node projects.

[Node.js Docs](#)

Runtime basics and server-side JavaScript learning.

[GitHub Docs: Hello World](#)

Repositories, branches, commits, pull requests.

Tip: Assign the linked docs as pre-read before the lab and ask students to note 3 learnings and 2 doubts.

End of Week 2 Teaching Deck

Next action: run the lab, collect GitHub links, review commits, and record weekly progress.

Student evidence to collect

Code commits, screenshots, working demo, README, report, reflection notes.

Thank you